

浙江大学

本科实验报告

课程名称:	B/S 体系软件设计
姓 名:	文豪
学 院:	计算机科学与技术学院
专 业:	软件工程
学 号:	3220104512
指导教师:	胡晓军

2024 年 12 月 20 日

浙江大学实验报告

课程名称: B/S 体系软件设计 实验类型: 综合
实验名称: 物联网设备管理平台
学生姓名: 文豪 专业: 软件工程 学号: 3220104512
同组学生姓名: / 指导老师: 胡晓军
实验地点: 线上 实验日期: 2024 年 12 月 20 日

文豪_3220104512_软件开发心得

引言

需求分析与系统设计

前端开发 (成就感)

后端开发 (享受)

数据库管理 (比较轻松)

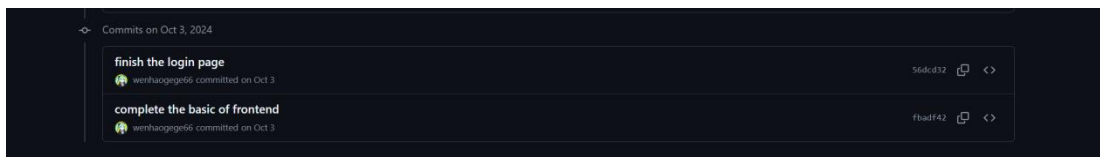
测试与部署 (极度痛苦)

开发中的挑战与收获

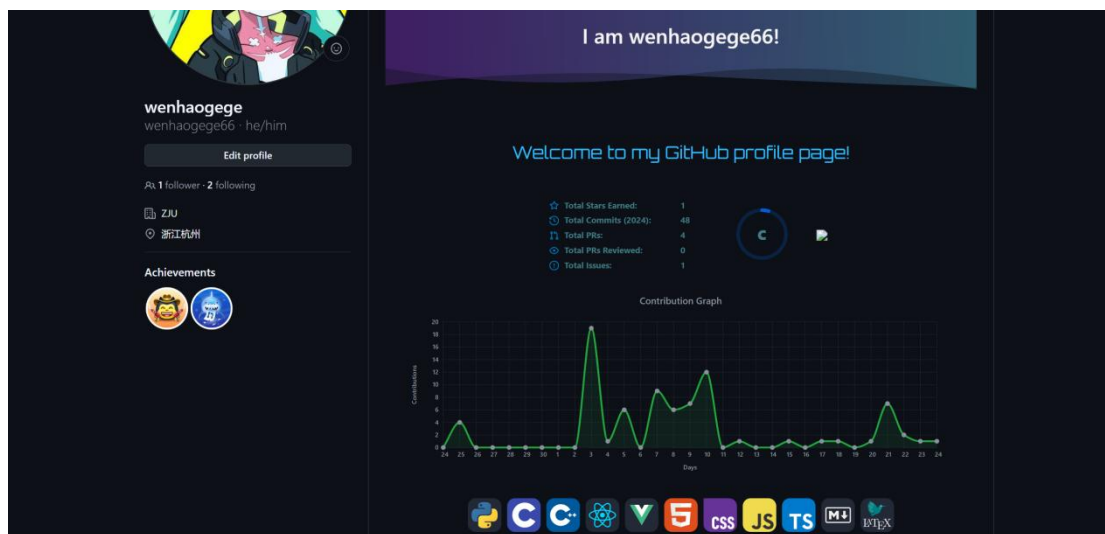
总结与展望

引言

本次开发的多平台商品比价系统是一次全栈开发的深入实践。通过这次项目,我从需求分析到系统设计,从前端到后端,从数据存储到 API 接口实现,从 vercel 部署到 docker 部署,全面参与了系统的开发过程。这既是一次全面技术能力的检验,也是我将课程知识转化为实际产品的成功尝试。本文记录开发过程中的思考、经验与心得。



10月3日开始项目搭建



github 个人主页

需求分析与系统设计

项目初期的需求分析尤为重要。在需求分析阶段，我首先明确了系统需要实现的基本功能，包括用户注册与登录、商品搜索与比价、价格提醒、购物车管理等。在需求规格与系统设计文档的撰写中，我细化了用户的交互流程，并设计了合理的用例图和数据库结构。

在系统设计中，我选择了 React 和 Django 的前后端分离架构，确保了项目的可扩展性与高性能。同时，通过引入 MySQL 作为数据库，以及 Selenium 爬取商品信息，为后续的数据处理与展现打下了坚实的基础。同时，我很早就开始学习使用 nginx，不仅仅是为后面部署做准备，更是为了将本地的一些文件和图片暴露出去。

前端开发（成就感）

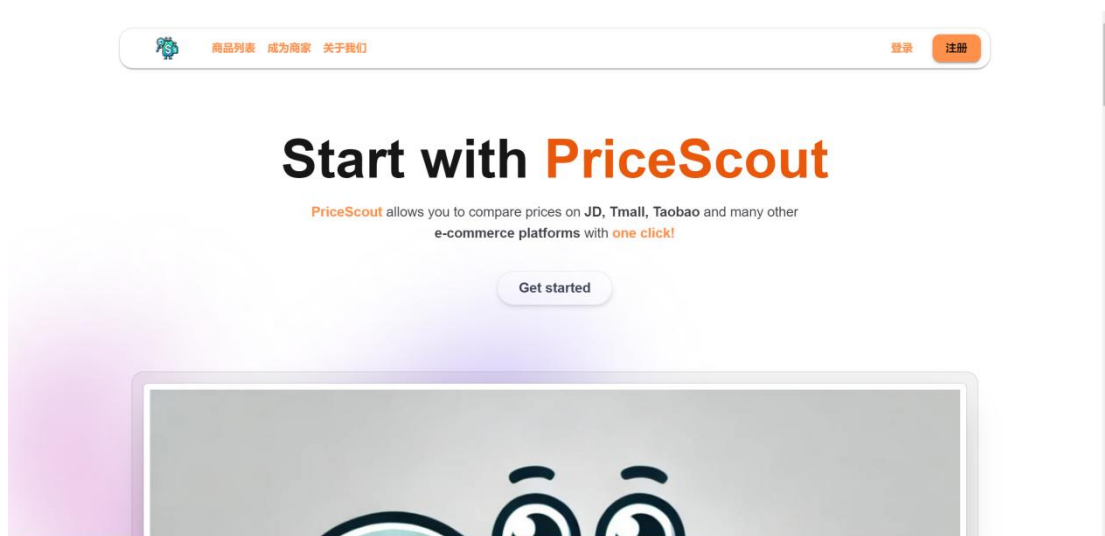
前端部分的开发主要使用了 React 和 MUI（Material-UI）组件库。为了提升用户体验，我特别注重了以下几点：

- 搜索推荐与商品显示：**通过动态展示搜索推荐列表，以及点击推荐时实时搜索对应商品，提高了系统的交互性。
- 商品卡片展示：**在商品比价界面，采用了卡片式布局，配合响应式设计，使得页面在不同设备上都能有良好的显示效果。

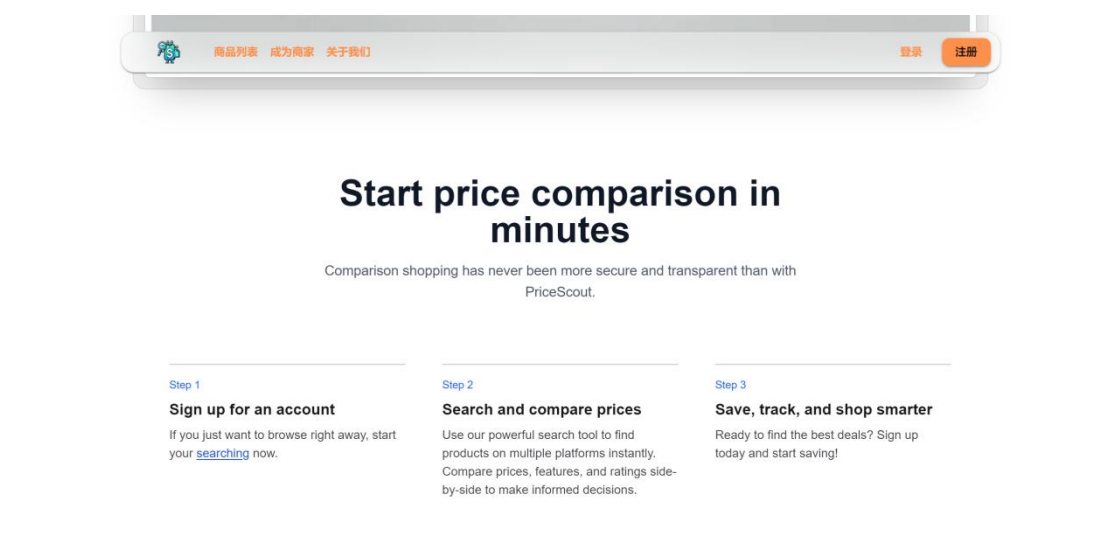
3. **状态管理**：利用 **React** 的状态管理机制，我实现了搜索、筛选等功能的无缝切换，并通过统一的组件设计减少了冗余代码。
4. **数据可视化**：商品价格趋势等功能通过图表清晰展示，增强了用户对数据的直观理解。

之所以选择 **React**，是因为之前和几个好朋友一起参加过一个区块链创新应用大赛，当时开发时发现 **react** 的组件库生态比 **VUE** 的要好不少，所以用的比较熟练，现在也继续选择用 **React**。

最让我自豪的自然是这个首页，下面其实还有我的一张“帅照”哈哈哈，也算是技术同学自己的浪漫了。

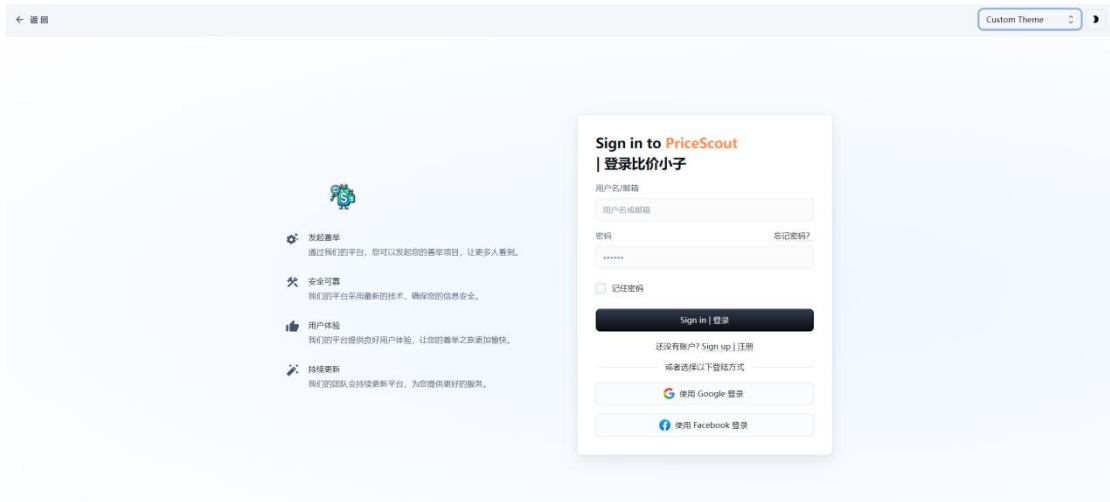


首页 1



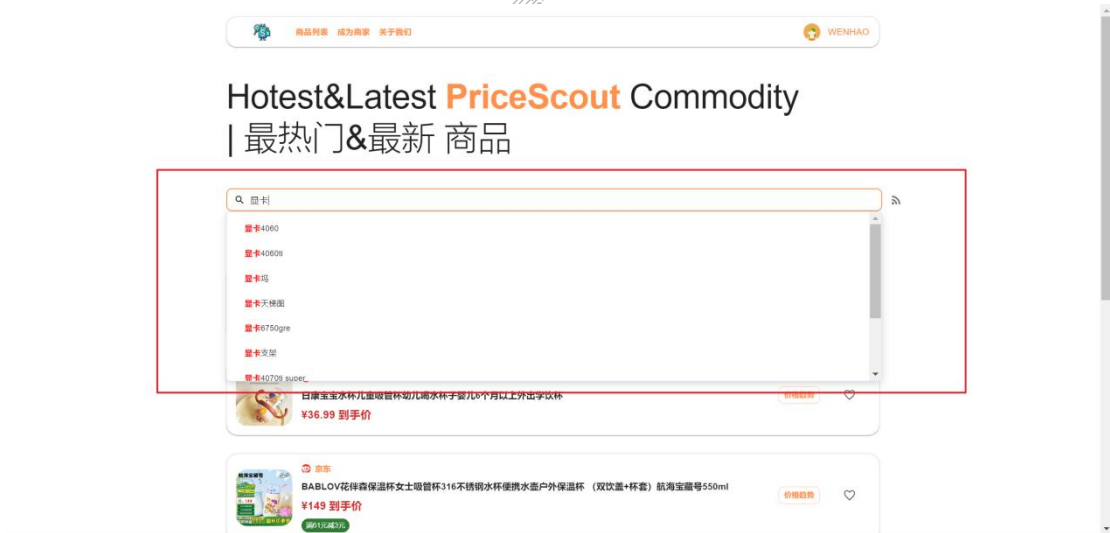
首页 2

之前在学习 Next.js 全栈开发的时候发现 MUI 这个组件库相当的好用，他的组件自定义属性也比较高，然后就比较耐心地模仿了一些商业网站的首页，做了这个比价网站的首页。



用户登录界面

商品搜索是整个项目的核心，当其他同学都在思考如何爬取多个网站的商品列表，如何让用户多次登录时，我直接利用后端修改请求头和源 IP 绕过跨域限制去调用一些现有服务器的 API。少了繁琐的登录后，可算是满足了我的实用主义心理。



商品搜索

历史价格趋势页面的 UI 我也是精心设计的，现在看起来真的很有成就感：



历史价格获取



价格趋势展示

后端开发（享受）

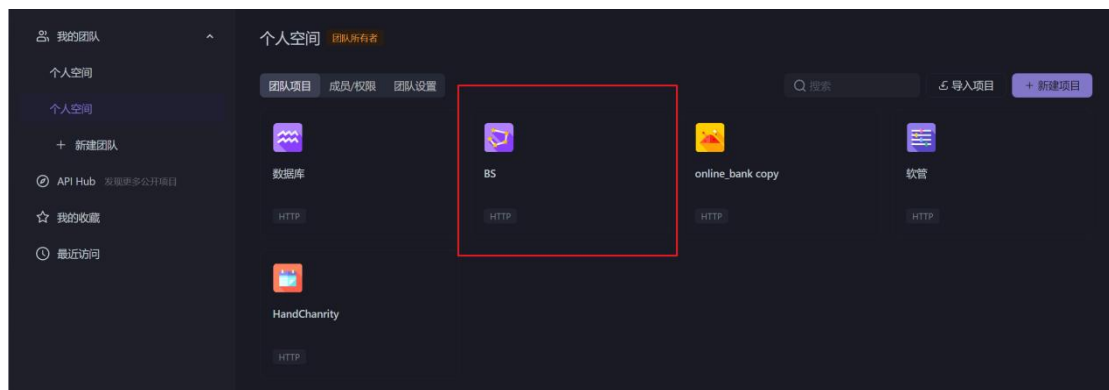
后端基于 Django 和 Django REST Framework 实现，主要负责以下功能：

- 1. 用户管理：**包括注册、登录、密码加密存储等功能，采用 SHA256 加密提升安全性。
- 2. 商品搜索与比价：**设计了灵活的查询接口，支持多条件筛选，返回实时更新的商品信息。
- 3. 数据爬取与存储：**使用 Selenium 定时抓取各电商平台的商品数据，保证了数据的及时性与准确性。

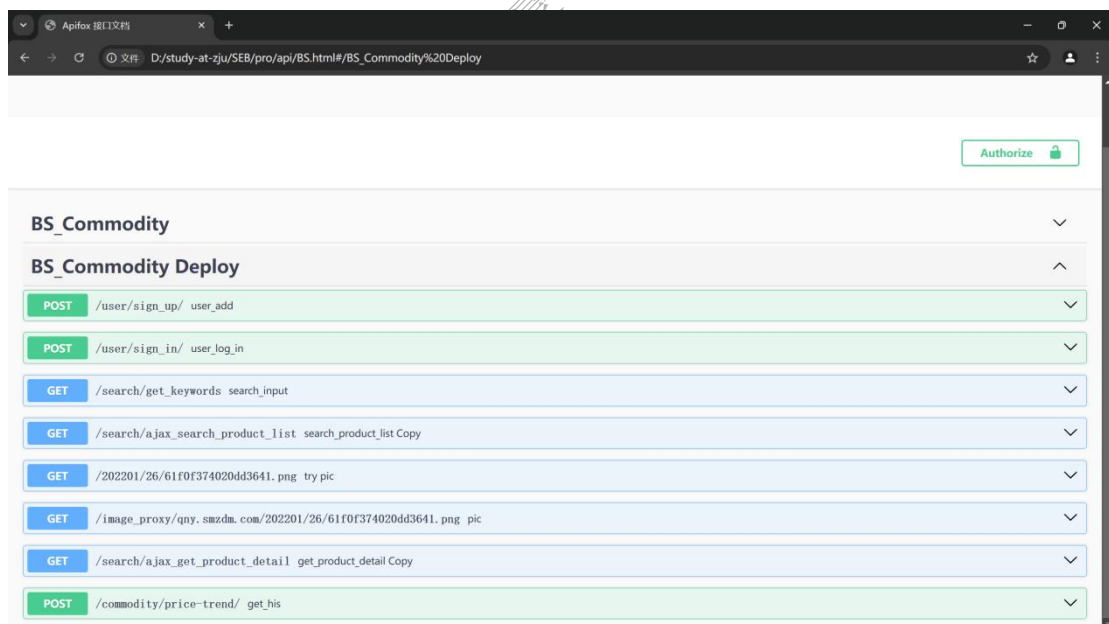
4. **降价提醒**：实现了价格阈值监控，当商品价格低于用户设定时，系统会发送通知。

后端之所以选择 Django，则是因为一年前上软件工程基础的时候，第一次小组合作开发网页的时候后端选择了 Django，学完后发现这简直太简单易懂了，从此就深深爱上了 django。

接口管理我选择 Apifox,也算比较有经验了，从此我的后端列表里又添一员大将：

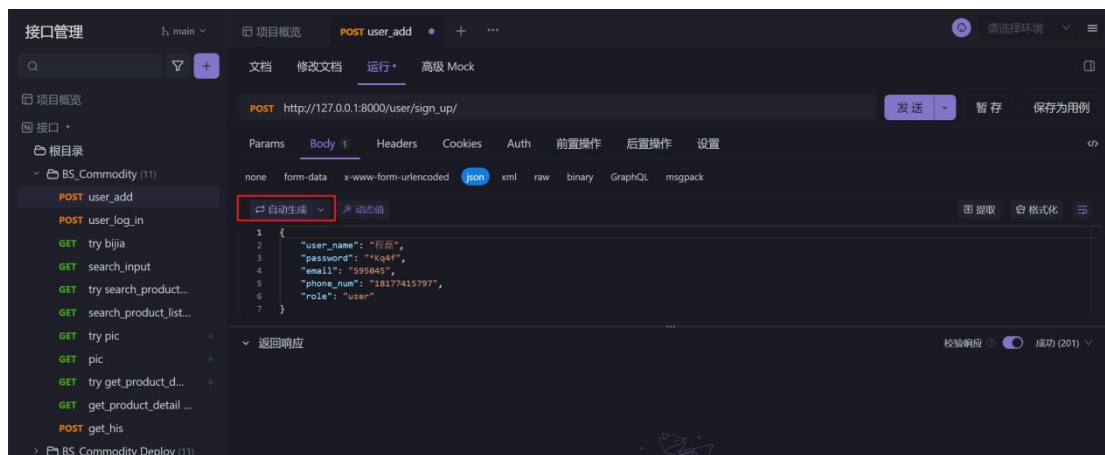


Apifox 项目管理



接口文档（部署版）

Apifox 的参数自动生成在调测和测试阶段都起了很大作用：



自动生成参数

数据库管理（比较轻松）

数据库的设计兼顾了效率与扩展性。在设计过程中，我特别关注了以下几点：

- **数据表规范化**：根据实体关系，设计了用户表、商品表、价格表等核心数据表，并通过 ER 图清晰展示了它们之间的关系。
- **索引优化**：为常用的查询字段添加索引，显著提高了搜索与比价功能的响应速度。
- **数据完整性**：通过外键约束和触发器，确保数据在插入与更新时的正确性。

测试与部署（极度痛苦）

（正经版）

在系统测试阶段，我设计了详细的测试用例，涵盖功能测试、性能测试和安全性测试。例如，使用 JMeter 对系统进行了压力测试，验证了系统在高并发情况下的稳定性。部署方面，我使用 Docker 将前后端与数据库容器化，简化了部署过程，并提升了系统的移植性。

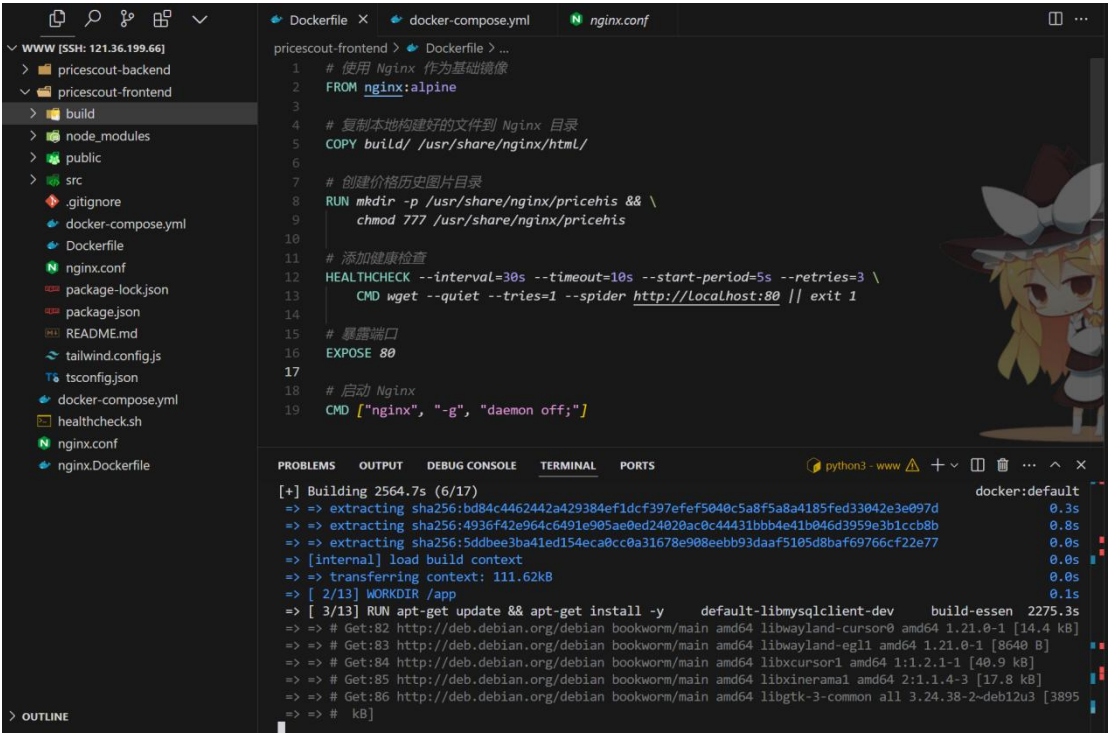
（非正经版）

首先是痛苦的 Docker 环境配置，我都已经记不清我换了多少次源，搜了多少条攻略了，我这玩意到现在都还不稳定，后来才知道国内之前大规模禁止过一次 Docker 源。之后可以考虑在服务器里搞个代理，但是我这服务器是免费领的 2C2G 华为云，等以后有钱了买个好的。

最开始的其实我只打算部署后端到一个云服务器上，然后前端用 vercel 单独部署

(因为我之前单独部署过前端，这个相当方便)，后来部署上去发现前端的地址变成公网的后，访问后端时由于浏览器安全规定不能加载 **http**，只能加载 **https**，可是我的图片是通过 **nginx** 暴露出来的，我再尝试了诸多办法后最终还是放弃了，选择在云服务器的 **Docker** 里部署一切。

在一次又一次的 **docker compose up --build** 中，我还学习到，**docker** 层的设置是如此精妙和重要，如果把经常需要更改的放到了下层，那么基本上每次重构都是从底层开始，加上服务器有宽带限制，直接极大地降低了效率。



```
1 # 使用 Nginx 作为基础镜像
2 FROM nginx:alpine
3
4 # 复制本地构建好的文件到 Nginx 目录
5 COPY build/ /usr/share/nginx/html/
6
7 # 创建价格历史图片目录
8 RUN mkdir -p /usr/share/nginx/pricehis && \
9     chmod 777 /usr/share/nginx/pricehis
10
11 # 添加健康检查
12 HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
13     CMD wget --quiet --tries=1 --spider http://localhost:80 || exit 1
14
15 # 暴露端口
16 EXPOSE 80
17
18 # 启动 Nginx
19 CMD ["nginx", "-g", "daemon off;"]
```

```
[+] Building 2564.7s (6/17)
=> extracting sha256:bd84c4462442a429384ef1dcf397efef5040c5a8f5a8a4185fed33042e3e097d 0.3s
=> extracting sha256:4936f42e964c6491e905ae0ed24020ac0c44431bbb4e41b046d3959e3b1ccb8b 0.8s
=> extracting sha256:5ddbee3ba41ed154eca0cc0a31678e908eebb93daaf5105d8baf69766cf22e77 0.0s
=> [internal] load build context 0.0s
=> transferring context: 111.62kB 0.0s
=> [ 2/13] WORKDIR /app 0.1s
=> [ 3/13] RUN apt-get update && apt-get install -y default-libmysqlclient-dev build-essen 2275.3s
=> # Get:82 http://deb.debian.org/debian bookworm/main amd64 libwayland-cursor0 amd64 1.21.0-1 [14.4 kB]
=> # Get:83 http://deb.debian.org/debian bookworm/main amd64 libwayland-egl1 amd64 1.21.0-1 [8640 B]
=> # Get:84 http://deb.debian.org/debian bookworm/main amd64 libxcursor1 amd64 1:1.2.1-1 [40.9 kB]
=> # Get:85 http://deb.debian.org/debian bookworm/main amd64 libxinerama1 amd64 2:1.1.4-3 [17.8 kB]
=> # Get:86 http://deb.debian.org/debian bookworm/main amd64 libgtk-3-common all 3.24.38-2-deb12u3 [3895
=> # kB]
```

网络不好——痛苦的等待

适配手机端

没想到这也能成为我的一大挑战，我最开始是只完成了网页版的版本，本以为手机端也就改几个地方的 **CSS**，但是发现有些地方几乎是需要重新写一个版本专门用于小屏幕的，甚至连字体大小都要重新斟酌，好在 **MUI** 组件库有比较多的参考资料。

开发中的挑战与收获

开发过程中遇到了许多技术难题，例如：

- **跨域问题**：通过配置 **Django** 的 **CORS** 中间件，**nginx** 的跨域设置，**django** 里 **API** 的 **@CSRF** 多次解决了前后端跨域请求的问题。

- **响应式设计**：在适配移动端时，我深入学习了 CSS Flexbox 与 Grid 布局，最终实现了良好的用户体验。
- **复杂数据处理**：商品信息的动态更新与多条件筛选，对数据处理与接口设计提出了较高的要求。

通过这次开发，我不仅熟悉了 React 与 Django 的技术栈，也深入理解了全栈开发的流程。从最初的需求分析到最终的系统交付，我学到了如何有效地协调前后端开发、优化用户体验，并确保系统的安全与稳定。

总结与展望

多平台商品比价系统的开发是一次全方位的技术挑战与提升的机会。通过这次项目，我完成了从单一角色到全栈开发者的角色转变。在未来，我希望能够继续优化系统的性能与功能，并将其扩展到更多应用场景。

同时，感谢课程组提供的宝贵学习机会，以及在开发过程中支持与帮助我的同学和老师。希望在未来的学习与工作中，能够继续提升自己的技术能力，为更多用户带来便利。

文豪 039